SIMATS ENGINEERING

SCENARIO-BASED ASSIGNMENT

Course: Artificial Intelligence | Course Code: CSA17

Course Outcome CO2

Experiment with search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently.

Submitted by: A.K. Kishore Kumar

Register Number: 1913260079

Answer booklet prepared according to the five scenarios in the assessment sheet.

1. AI-Powered Drone Delivery in a Flood-Affected Region

i. Greedy Best-First Search

Greedy Best-First Search chooses the next location with the smallest heuristic value, here the straight-line distance to the medicine delivery point. It can quickly select a route that appears close to the destination, which is useful when time is critical. However, it considers the estimated distance rather than the full travel cost. A short route may contain flooded roads, dangerous weather, or difficult terrain. With incomplete updates, the drone may choose an unsafe or blocked path and need to search again.

ii. A* Search

A* evaluates each route using **f(n) = g(n) + h(n)**, where g(n) is the actual cost accumulated so far and h(n) is the estimated remaining cost. The actual cost can include travel time, weather, terrain difficulty, and risk. The heuristic estimates the remaining distance or time. A* therefore balances the route already travelled with the estimated cost to the destination. With non-negative costs and a suitable admissible heuristic, A* can find an optimal route in a static map; changing conditions may require replanning.

iii. Effect of heuristic accuracy

A realistic heuristic guides both algorithms toward promising routes. If the straight-line estimate is misleading because of rivers, blocked roads, or severe weather, Greedy Search may be strongly misdirected because it relies on h(n) alone. A* also becomes less efficient when h(n) is poor, but g(n) helps it account for the costs already incurred. An admissible heuristic that does not overestimate the remaining cost supports A* optimality in the appropriate search setting.

iv. Effect of blocked paths and dynamic changes

When a path becomes blocked, the search must update the map, remove or penalize the unavailable route, and search again. Repeated changes can cause extra computation and delay. A practical drone system should receive frequent updates, check route safety before movement, and replan from its current position rather than blindly follow an old route.

v. Recommendation

A* is the better default when safe, low-cost routing matters because it considers both accumulated cost and estimated remaining cost. For strict real-time operation, it should be combined with rapid replanning, current obstacle information, and a safety rule that rejects dangerous routes. If updates are extremely frequent and computation is limited, a real-time or incremental A* variant may be preferable. The drone should prioritize safety over the shortest distance.

2. Smart-City Traffic Signal Optimization

Formulation as an optimization problem

A solution is a set of signal timings and phase sequences for intersections. The objective is to minimize a weighted total such as J = w1(waiting time) + w2(fuel consumption) + w3(congestion), where the weights represent the city's priorities. Constraints include minimum and maximum green times, safe phase order, pedestrian crossing time, and coordination between nearby intersections. Traffic sensors provide changing inputs, so the system must repeatedly optimize using recent data.

Why traditional search can be inefficient

The number of possible timing combinations grows rapidly with the number of intersections and signal phases. Exhaustively evaluating every configuration is too expensive, and traffic conditions change before a full search may finish. Heuristics and optimization methods can examine promising candidates without exploring every possible state.

i. Hill Climbing and limitations

Hill Climbing starts with a timing plan and makes a small change, such as extending a green phase, if that change improves the objective. It repeats until no neighboring plan appears better. It is simple and fast, but it may stop at a

local optimum and does not naturally explore distant alternatives.

ii. Local maxima, plateaus, and ridges

A **local optimum** is a plan that is better than nearby plans but not the best overall—for example, reducing waiting at one junction while worsening a connected corridor. A **plateau** occurs when many timing changes give nearly the same score, leaving the algorithm without a clear direction. A **ridge** occurs when improvement requires a coordinated sequence of changes across several intersections; changing only one signal at a time may not help.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse solution early in the search, allowing it to escape local optima. The probability of accepting worse changes decreases as the temperature parameter is reduced. A Genetic Algorithm maintains a population of timing plans and uses selection, crossover, and mutation to create new candidates. It explores multiple parts of the search space, although it needs careful parameter tuning and computational resources.

iv. Recommendation

A Genetic Algorithm is suitable for offline or periodic optimization across many connected intersections because it can explore diverse timing plans and optimize several objectives. For rapid online adjustment, a hybrid approach is stronger: use a Genetic Algorithm to develop coordinated plans, then apply a fast local method to fine-tune timings as traffic changes. The system must enforce safety constraints and continuously monitor performance.

3. Autonomous Mars Rover in Unknown Terrain

i. Why an online search agent is required

An offline search agent assumes that it has a sufficiently complete map and model before planning. The rover begins with incomplete information and discovers hazards as it moves. An online search agent alternates between sensing, acting, learning, and planning, so it can choose actions using the information currently available and adapt as the map improves.

ii. Partial observability and dynamic-environment challenges

The rover's sensors may have limited range or accuracy, and rocks, craters, slopes, or loose soil may be hidden. It cannot know every route cost in advance. Communication delays and limited power or processing capacity also restrict decisions. A wrong estimate can cause damage or waste valuable mission time, so the rover needs uncertainty-aware planning and conservative safety margins.

iii. Building and updating an internal model

The rover combines sensor readings, camera images, wheel/position estimates, and observations from previous movement to maintain a local map. It labels known free areas, obstacles, uncertain regions, and estimated terrain costs. After each action it compares the predicted result with the observed result, updates its map and confidence, and replans if a route is unsafe or less promising.

iv. Comparison with uninformed and informed search

Uninformed methods such as Breadth-First Search or Uniform-Cost Search use no goal-directed heuristic, and may explore many states before finding a route. Informed methods such as A* use a heuristic to focus the search toward the target, but generally work best when a useful map and cost estimates are available. Online search is distinguished by learning through action in an initially unknown environment; it can use informed or uninformed planning within that cycle.

v. Recommended approach

Use an online, incremental planning approach supported by A*-style heuristic guidance on the rover's evolving map. The rover should move cautiously into uncertain areas, maintain safe distance from hazards, and replan after new observations. This combines adaptability with goal-directed efficiency while keeping safety as the main constraint.

4. University Exam Timetabling as a Constraint Satisfaction Problem (CSP)

Formulation

A CSP consists of variables, domains, and constraints. The goal is to assign a valid time and hall to every exam while satisfying all hard constraints.

i. Variables, domains, and constraints

Variables: one variable for each course exam. Its assigned value can be a time slot and examination hall.
Domains: the available time slots and halls that can accommodate each exam, taking account of room capacity, accessibility, and faculty availability.
Hard constraints: exams taken by the same student cannot overlap; a hall cannot host two exams at the same time; hall capacity must be sufficient; faculty must be available; prerequisite ordering rules must be respected; and required accommodations must be provided.
Soft preferences: where possible, avoid placing many difficult exams on the same day or creating inconvenient gaps for students.

ii. Backtracking search

Backtracking assigns a value to one exam variable at a time. If an assignment violates a constraint, the algorithm rejects it and tries another value. If no value works, it returns to an earlier variable and changes that assignment. Search can be improved by choosing the most constrained exam first and trying the least-constraining time slot first.

iii. Constraint propagation and forward checking

After assigning an exam, forward checking removes incompatible values from the domains of unassigned exams. For example, if a course is placed in a slot, that slot is removed from the domains of exams sharing students or requiring the same hall. If any domain becomes empty, the algorithm detects the conflict immediately and backtracks, avoiding many unnecessary steps.

iv. Real-world challenges and scalable strategy

Large universities have many courses, shared students, limited rooms, changing faculty availability, and special accommodations. Late changes can invalidate earlier assignments. A practical strategy combines constraint propagation, minimum-remaining-values variable selection, degree heuristics, and least-constraining-value ordering. For very large instances, the timetable can be solved in stages or with optimization methods, followed by validation against every hard constraint.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

i. How Minimax models the problem

Minimax represents the game as a tree of states and legal moves. The AI is the maximizing player and the opponent is the minimizing player. Terminal states receive utility values, such as win, draw, or loss. Values are propagated upward: a maximizing node selects the highest child value, while a minimizing node selects the lowest. The AI chooses the move with the best worst-case outcome, assuming an optimal opponent.

ii. Computational challenges

If the average branching factor is b and the search depth is d, basic Minimax may examine approximately O(b^d) nodes. The number of states grows exponentially with depth, making deep searches costly. Real-time games also have strict decision deadlines and may include simultaneous events, incomplete information, or a large number of possible actions.

iii. Alpha-Beta pruning

Alpha-Beta pruning produces the same Minimax decision while skipping branches that cannot affect the final choice. Alpha is the best value found so far for the maximizing player; beta is the best value found so far for the minimizing player. When alpha is greater than or equal to beta, the remaining children of that node can be pruned

because the opponent would already choose a better alternative elsewhere. Good move ordering increases pruning. In the best case, the effective search can approach O(b^(d/2)), allowing deeper analysis in the same time.

iv. Example evaluation function

A practical heuristic evaluation can be:
Score(s) = 5 × resource advantage + 4 × unit strength + 3 × territory control + 2 × objective progress – **5 × threat to critical units** – **3 × resource shortage.**
Each term should be normalized to a comparable scale and its weight tuned through testing. Resource advantage measures economic strength; unit strength measures combat capability; territory and objectives reflect strategic position; threats and shortages penalize vulnerable or unsustainable states. Terminal wins and losses should receive much larger positive or negative scores than ordinary positions.

v. Recommendation for real-time performance

Use depth-limited Minimax with Alpha-Beta pruning, strong move ordering, and a fast evaluation function. Iterative deepening searches depth 1, then 2, and so on; if the deadline arrives, the AI can return the best move from the last completed depth. Time limits, caching of repeated states, and selective search can further improve speed. This provides a practical balance between decision quality and response time.

End of Assignment